PROJETO PRÁTICO AVALIATIVO

API de Gerenciamento de Pedidos

Tópicos Avançados em Sistemas de Informação II

1. Objetivo

- [ ] Desenvolver uma API REST para gerenciamento de pedidos de uma pequena loja, aplicando arquitetura em camadas, validação de dados, tratamento de erros, versionamento com Git e Github.
- [ ] O foco da avaliação não é apenas entregar código funcional, mas demonstrar compreensão das decisões técnicas e capacidade de testar o comportamento implementado.

2. Tecnologias e limites do projeto

- [ ] Node.js e Express, utilizando JavaScript.
- [ ] Organização mínima: routes, controllers e services; pastas adicionais quando justificadas.
- [ ] Armazenamento em memória permitido. Banco de dados não é obrigatório.
- [ ] Jest para testes automatizados; Postman para testes manuais; GitHub e GitHub.
- [ ] O projeto deverá iniciar com npm install e npm start (ou comando documentado no README).

3. Requisitos funcionais

Código	Requisito
RF01	Cadastrar pedido com cliente, tipo, valor e prioridade.
RF02	Listar os pedidos cadastrados.
RF03	Consultar um pedido pelo identificador.
RF04	Atualizar o status de um pedido.

4. Endpoints obrigatórios

Método	Rota	Resultado esperado
POST	/pedidos	Criação de pedido válido: HTTP 201
GET	/pedidos	Lista de pedidos: HTTP 200
GET	/pedidos/:id	Pedido existente: 200; inexistente: 404
PATCH	/pedidos/:id/status	Atualização válida: 200; dados inválidos: 400; ID inexistente: 404

Exemplo de requisição de criação:

```
POST /pedidos
{
  "cliente": "Carlos Silva",
  "tipo": "entrega",
  "valor": 350,
  "prioridade": "normal"
}
```

Exemplo de resposta:

```
{
  "id": 1,
  "cliente": "Carlos Silva",
  "tipo": "entrega",
  "valor": 350,
  "prioridade": "normal",
  "status": "recebido"
}
```

Para atualização de status, utilize um corpo JSON como {"status":"cancelado"}. Estados básicos sugeridos: recebido, em_preparacao, enviado, entregue e cancelado. A aplicação deve validar transições conforme as regras que implementar e documentar.

5. Validações e respostas de erro

- ☐ Cliente obrigatório: texto não vazio, inclusive após remoção de espaços nas extremidades.
- ☐ Valor deve ser numérico e maior que zero.
- ☐ Tipo e prioridade devem ser valores reconhecidos e documentados no README.
- ☐ Identificadores inexistentes devem produzir HTTP 404.
- ☐ Dados inválidos devem produzir HTTP 400, com mensagem JSON explicativa.
- ☐ Erros inesperados não devem ser apresentados como validações do usuário; devem resultar em tratamento apropriado (ex.: HTTP 500).

6. Evidências obrigatórias — Postman e GitHub

Entregar uma pasta evidencias/ no repositório, com capturas legíveis que exibam método, URL, corpo da requisição quando aplicável, resposta e código HTTP:

Arquivo sugerido	Cenário
01-criacao.png	POST válido com resposta 201.
02-validacao.png	POST inválido com resposta 400.
03-listagem.png	GET da lista de pedidos.
04-consulta-inexistente.png	GET de ID inexistente com 404.
05-atualizacao.png	PATCH de status válido.

Os prints são evidências complementares, não prova isolada de autoria. O professor poderá executar a API e aplicar entradas diferentes das apresentadas nas capturas.

7. GitHub e documentação

- ☐ Entregar o link do repositório GitHub com o código-fonte, package.json, evidências.
- ☐ Manter histórico de desenvolvimento com pelo menos cinco commits significativos. Commits isoladamente não comprovam autoria.

- [ ] Incluir README com instruções de instalação e execução, rotas, exemplos, arquitetura, padrão implementado e testes.
- [ ] Não incluir node_modules, senhas, tokens ou arquivos .env com credenciais.

8. Registro do uso de inteligência artificial

É permitido utilizar IA como apoio. O aluno permanece responsável por compreender, testar, justificar e corrigir todo o código entregue. No README, incluir uma seção “Uso de IA”, informando se utilizou, para quais tarefas e descrevendo ao menos uma sugestão aceita, modificada ou rejeitada, com justificativa.

9. Critérios de avaliação

Critério	Pontuação
API e requisitos funcionais	0,5
Arquitetura em camadas	0,5
Evidências Postman	0,5
Git e documentação	0,5
TOTAL	2,0

A atividade individual será realizada sem apresentação oral e avaliará interpretação de código, localização de responsabilidades, previsão de comportamento e pequenas correções. Sua data e formato serão comunicados pelo professor.

10. Entrega e verificação

- [ ] Entregar o link do repositório e o RA do aluno
- [ ] O professor poderá aplicar testes adicionais com dados não apresentados nos exemplos, respeitando os requisitos publicados.
- [ ] Inconsistências entre código, prints, README e execução real poderão reduzir a pontuação dos critérios correspondentes.
- [ ] A avaliação considerará funcionamento e domínio conceitual, não a simples quantidade de arquivos ou linhas de código.